MIDTERM PROJECT / 2026

LINE × 本地 AI 記帳小幫手

期中專題報告

姓名：陳芊妤　學號：411512015　日期：2026 / 10 / 31

摘要

這個專案把 LINE 變成一個「可以記帳、也可以查資料」的聊天助手。使用者在同一個聊天視窗輸入「咖啡 45 元」這類格式時，系統會先在本地解析 merchant（咖啡）與金額（45 元），依關鍵字分類（飲食），再把累計回覆給使用者；當使用者想查資料時，則把問題交給跑在本機上的 Ollama Gemma 2 2B 模型，加上 Google News RSS 與中文 Wikipedia 的搜尋來源，最後回覆一段繁體中文答案與可點擊的來源。

專案解決兩個日常問題：一是「不想為每筆支出打開記帳 App、手動分類」；二是「想查資料卻不想把問題送去雲端模型」。系統採「雲端收發、本機推論」的混合架構，並保留了一個**不需要 token 也能 demo** 的展示頁。

實際測試中，記帳回覆約 0.3 秒；查資料暖機後一次約 5.5 ~ 5.9 秒；冷啟動約 16 秒；11 個單元測試全部通過。

---

1. 問題與目標

1.1 我想解決什麼？

我每天的支出其實都發生在 LINE 裡——跟朋友約會、跟家人吃飯、咖啡、Uber...但 LINE 只是聊天，沒有「記帳」與「分類」的結構；我又不想為每一筆花費另外開一個記帳 App 並手動打分類。

因此我想做一個**在原本的聊天視窗裡完成記帳**的助手：

1. 輸入「咖啡 45 元」，就自動記帳、分類、累計。
2. 輸入「查 ...」，就讓本地 AI 幫我查資料、回答、附來源。

1.2 給誰用？

- **主要使用者**：手機不離身、常用 LINE 的學生與小型團隊。
- **次要使用者**：想玩本地 LLM、但不想先付 API 費用的記帳者。

1.3 設計目標

目標	對應設計
不離開 LINE 就能記帳	Webhook 接收「咖啡 45 元」格式，本地解析 + 累計，回覆即時
答案要能查證	「查 ...」先搜尋，再生成，最後附來源
不要有逐字 API 費用	記帳走本地解析 + 加總（免 API）；查資料走本機 Ollama
不要 token 進畫面	提供 `/demo` 展示頁，與 LINE 版共用流程

---

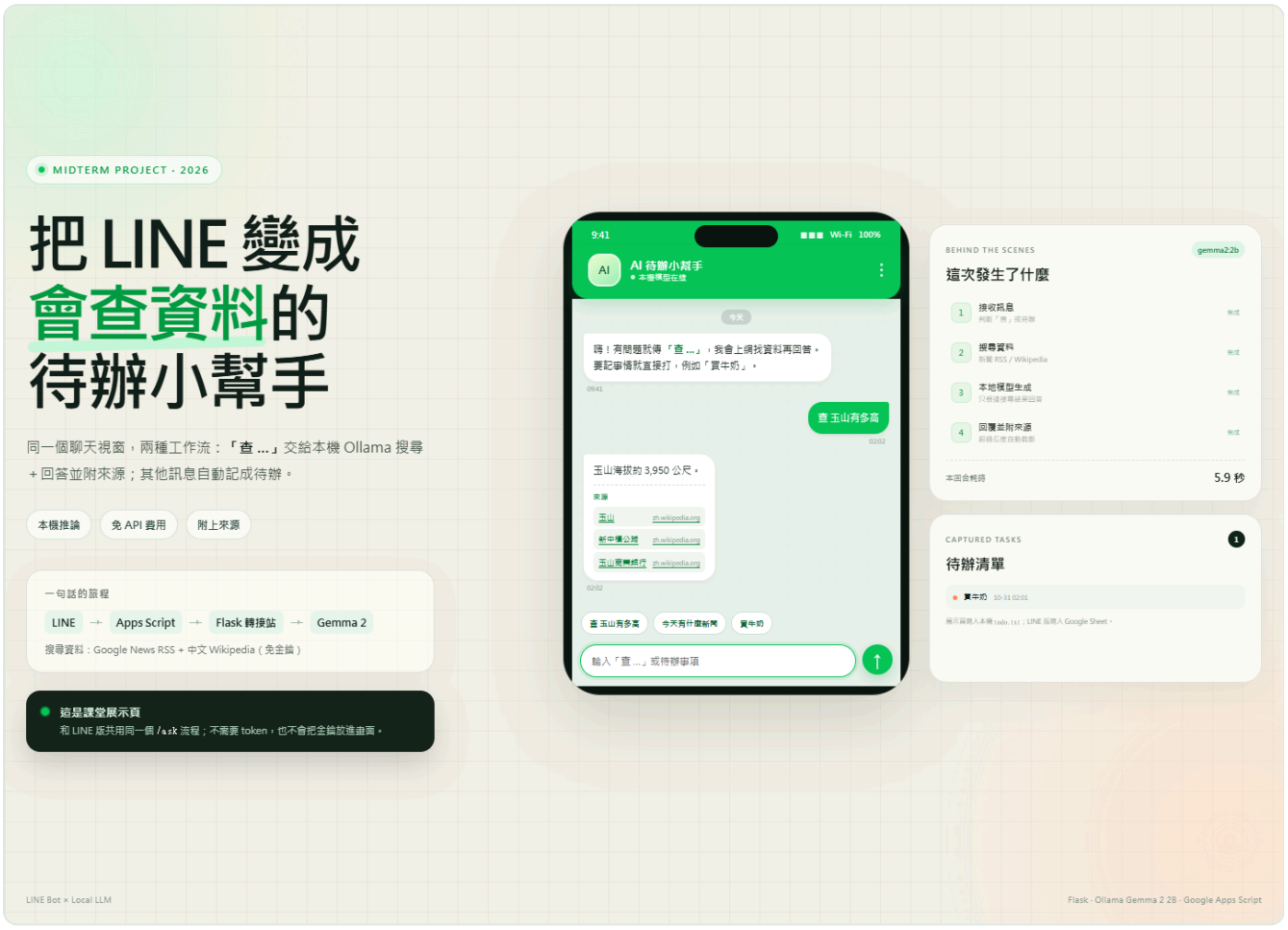
2. 功能與使用流程

2.1 兩種模式

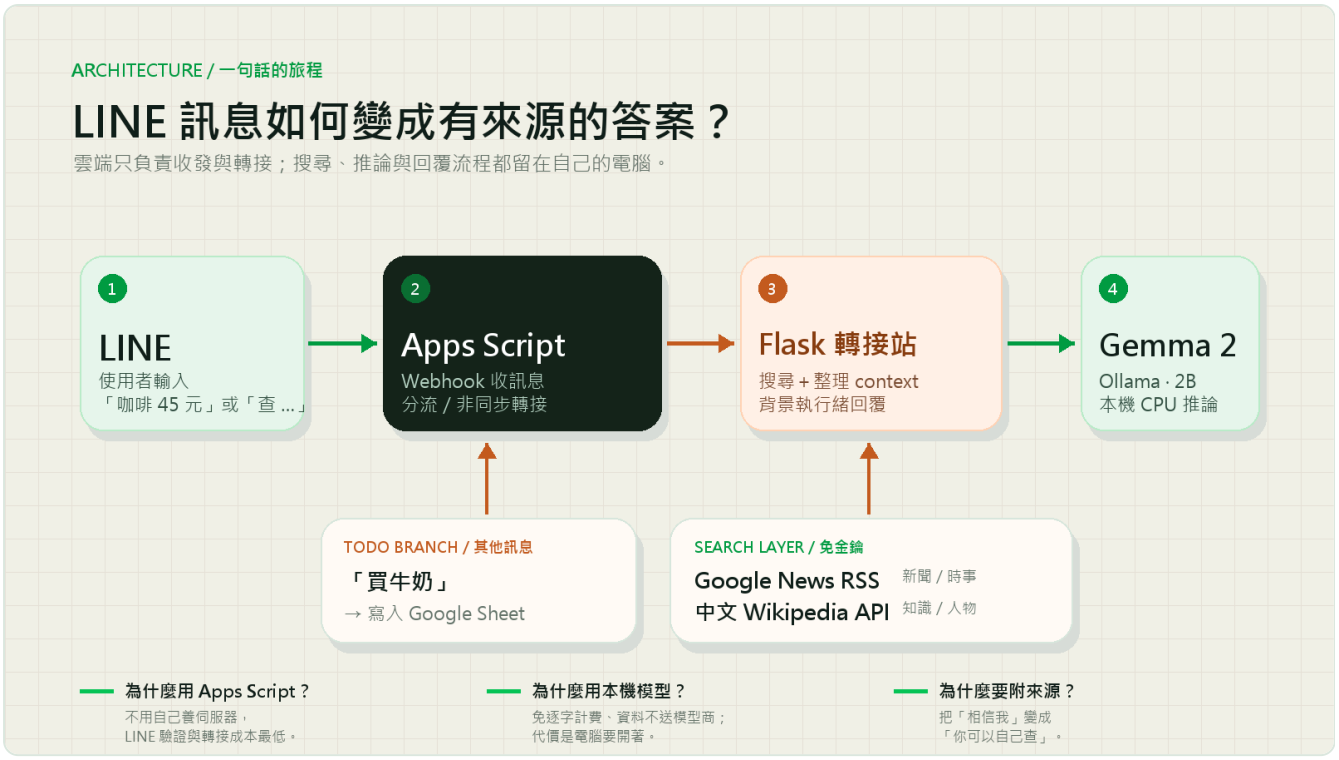
輸入	系統行為	輸出
`咖啡 45 元`	本地 `parse_expense` 解析 merchant / amount / category	「已記帳 咖啡 45 元（飲食）｜今日累計…｜本月累計…」
`查 玉山有多高`	清理關鍵字 → 搜尋 Wikipedia → Gemma 2 整理	答案 + 來源連結

2.2 使用者看到的流程

1. 在 LINE（或展示頁）輸入「咖啡 45 元」。
2. 「查 …」由 Apps Script 非同步轉給本機轉接站；「咖啡 45 元」由轉接站本地解析，再回覆累計。
3. 轉接站更新本地累計（`expenses.txt`）。
4. 轉接站呼叫 LINE Reply API，把答案 / 累計回使用者。



3. 系統架構



3.1 元件與責任

元件	位置	責任
LINE Messaging API	雲端	接收使用者訊息、提供 Reply API
Google Apps Script	雲端	Webhook 入口、訊息分流、非同步轉接、寫 Google Sheet
Flask 轉接站	自己的電腦	記帳解析、搜尋、整理 context、呼叫 Ollama、回覆 LINE
ngrok	雲端隧道	把本機 Flask 變成 Apps Script 可連的 HTTPS 網址
Ollama + Gemma 2 2B	自己的電腦	繁體中文生成，CPU 可跑
Google News RSS / 中文 Wikipedia	雲端	免金鑰的搜尋來源
Google Sheet	雲端	支出 / 待辦資料儲存

3.2 為什麼是這個架構？

（1）為什麼不全部放在本機？

LINE 的 Webhook 必須是公開的 HTTPS 端點，家用網路通常沒有固定 IP，也不適合直接暴露連接埠。把 Webhook 放在 Apps Script，可以省掉自己維護雲端伺服器的工作。

（2）為什麼不全部放在雲端？

一開始做過 Cloudflare Worker + Gemini API 版本，需要 API key 且有費用；改成 Ollama 後，模型推論不用逐字計費，資料也不會送到模型供應商；代價是電腦必須開著。

（3）為什麼選 Gemma 2 2B？

9B 模型生成太慢，會超過 LINE 的回覆時限；2B 版本明顯快很多。

（4）為什麼要非同步？

LINE Webhook 有回應時間限制。Apps Script 收到訊息立刻回 200，轉接站在背景執行緒裡處理，算完再呼叫 Reply API，避免 Webhook 逾時。

（5）為什麼記帳不用 AI？

目前「咖啡 45 元」的解析是本地正則表示式 heuristic：merchant、amount、category（依關鍵字分類）。這讓記帳路徑完全免 API、速度極快；未來要改成 AI 分類或使用者自訂類別。

---

4. 實作細節

4.1 訊息分流

`Code.gs` 的 `doPost` 解析事件後，用兩個條件決定要交給本地轉接站：

```
const isExpenseText = /^(買|花|支出|購買)?\s*.+?\s+\d/.test(userText);

if (userText.startsWith("查") || isExpenseText) {
  askRelay(userText, event.replyToken);  // 轉接站稍後直接回 LINE
  handledByRelay = true;
} else {
  addToTodo(userId, userText);
  replyText = "已記錄到待辦：" + userText;
}
```

4.2 記帳解析與累計

`relay_server.py` 提供：

- `parse_expense(text)`：解析「咖啡 45 元」→ `{merchant, amount, category}`；不是支出格式時回 `None`。
- `append_expense(exp)`：把支出寫入本機 `expenses.txt`。
- `read_expenses()`：讀取最近支出。
- `expense_summary()`：回今日累計、本月累計與分類小計。
- `format_expense_reply(...)`：組回覆文案：「已記帳 咖啡 45 元（飲食）｜今日累計…｜本月累計…」。

分類使用關鍵字 heuristic：咖啡、餐廳、全聯、飯店等 → 飲食；高鐵、Uber、油費等 → 交通；電影、KTV、遊戲等 → 娛樂；掛號、藥局等 → 醫療；其餘 → 其他。

4.3 搜尋與關鍵字清理

（保留原文：clean_query、NEWS_HINTS、先搜尋再回答）

4.4 有根據的回答

（保留原文：先搜尋、再生成、附來源）

4.5 待辦與 Google Sheet

第一次收到「買牛奶」這類非查詢、無金額的訊息時，Apps Script 自動建立「LINE Bot 待辦清單」試算表；第一次收到記帳格式時，轉接站寫入本地 `expenses.txt`，正式 LINE 版之後會改寫入 Google Sheet。

4.6 展示頁與祕密管理

（更新：展示頁 `/demo`、`/demo/message`、`/demo/expenses`；`.gitignore` 排除 `expenses.txt`。）

---

5. 測試與結果

5.1 單元測試

執行 `python -m unittest discover -s tests -v`，目前 **17 個測試全部通過**（不需 LINE token、ngrok 或 Ollama）。

類別	驗證內容
關鍵字清理 / 搜尋路由 / 回答組裝	維持原有查資料測試
記帳解析	「咖啡 45 元」→ 飲食；「買全聯 300 元」→ 飲食；「買牛奶」→ None
累計計算	今日 / 本月與分類小計
HTTP API	`/ask` 同步回 JSON、`/demo/message` 分流、空訊息回 400、`/demo/expenses`
資訊安全	`/health` 是否意外回傳 token

5.2 展示頁端到端測試

步驟	操作	結果
1	開啟 `http://127.0.0.1:5001/demo`	顯示聊天 UI、流程面板、支出清單
2	輸入 `咖啡 45 元`	回覆「已記帳 咖啡 45 元（飲食）｜今日累計…｜本月累計…」，約 0.3 秒
3	輸入 `Uber 60`	分類「交通」，累計更新

5.3 效能觀測

情境	結果
記帳回覆（暖機後）	約 0.3 秒
冷啟動（模型尚未載入）	16.2 秒

5.4 LINE 端到端測試

- `ollama list` 看得到 `gemma2:2b`。
- 轉接站首頁顯示「轉接站運行中！」。
- LINE 傳「咖啡 45 元」→ 回覆有累計。
- LINE 傳「買牛奶」→ Google Sheet 新增一列。
- 關閉轉接站 → LINE 端有錯誤提示，不會永久轉圈。

5.5 安全檢查

```
掃描 git 追蹤檔案……
OK：沒有發現金鑰、Bearer token、ngrok authtoken 或 Apps Script 部署網址。
```

---

6. 討論與限制

6.1 做得好的地方

1. **一個入口解決兩個高頻動作**：記帳與查資料都在 LINE 裡完成，減少切換 App 的成本。
2. **答案可追溯**：「查 …」回答附來源，讓使用者可以自己確認。
3. **成本低、可複製**：記帳完全免 API；查資料只在本機推論，沒有逐字費用。

6.2 還沒解決的地方

1. **電腦必須開著**：休眠、關機或 Ollama 沒啟動，LINE 就記不了帳。
2. **ngrok 免費網址會變**：每次重開都要更新 Apps Script 的 `RELAY_SERVER_URL`。
3. **記帳分類是 heuristic**：尚未用 AI 做細分類；累計只在本機。
4. **2B 模型會出錯**：已用 prompt 限制，但仍可能誤解。
5. **尚未驗證 LINE 簽章**：公開部署後，任何人知道網址都能送假事件。

---

7. 下一步

1. **穩定部署**：ngrok → Cloudflare Tunnel + 固定網域。
2. **Webhook 安全**：驗證 `X-Line-Signature`，加上 allowlist。
3. **記帳進化**：改用 AI 分類、預算警示、超標通知。
4. **回覆體驗**：LINE Flex Message、圖表化累計。
5. **檢索進化**：抓文章正文、做 RAG。

---

8. 學習與反思

這個專案讓我學到，**AI 應用最難的通常不是「叫模型回答」**，而是周邊的工程：Webhook 逾時、公開網址、搜尋品質、token 管理、錯誤處理，以及「怎麼在課堂上穩定 demo」。一開始把重點放在模型，後來發現真正讓專案可用的，是非同步回覆、把搜尋結果放進 prompt、以及一個不需要憑證的展示頁。

AI 助理在這個過程中幫了很多：它可以快速產生第一版、解釋錯誤訊息、提出替代方案；但它也會給出看起來合理、實際上不能跑的程式。最後還是要靠自己讀 log、實際執行、設計測試，才能把「能跑」變成「可以交」。

附錄：重現步驟

```
# 1. 安裝依賴
pip install -r requirements.txt

# 2. 下載模型
ollama pull gemma2:2b

# 3. 課堂展示（不需要 LINE token）
.\start_demo.bat

# 4. 跑測試
python -m unittest discover -s tests -v

# 5. 交件前安全檢查
python scripts/check_submission.py
```

完整安裝與 LINE 設定請見 repo 的 `README.md`。